

Docker CLI Cheat Sheet

Commands	Brief Description
<code>docker info</code>	Display system information
<code>docker version</code>	Display the system's version
<code>docker login</code>	Log in to a Docker registry
<code>docker pull [imageName]</code>	Pull an image from a registry
<code>docker run [imageName]</code>	Run containers
<code>docker run -d [imageName]</code>	Detached mode
<code>docker start [containerName]</code>	Start stopped containers
<code>docker ps</code>	List all running containers
<code>docker ps -a</code>	List all running and stopped containers
<code>docker stop [containerName]</code>	Stop containers
<code>docker kill [containerName]</code>	Kill containers
<code>docker image inspect [imageName]</code>	Get image info
<code>docker run -memory="256m" nginx</code>	Max Memory
<code>docker run -cpus=".5" nginx</code>	Max CPU
<code>docker run -it nginx /bin/bash</code>	Attach shell
<code>Docker run -it microsoft/powershell:nanoserverpwsh.exe</code>	Attach powershell
<code>docker container exec -it [containerName] bash</code>	Attach to a running container
<code>docker rm [containerName]</code>	Removes stopped containers
<code>docker rm \$(docker ps -a -q)</code>	Removes all stopped containers
<code>docker images</code>	Lists images
<code>Docker rmi [imageName]</code>	Deletes the image
<code>docker system prune -a</code>	Removes all images not in use by any containers
<code>docker build -t [name:tag] .</code>	Builds an image using a Dockerfile located in the same folder
<code>docker build -t [name:tag] -f [filename]</code>	Builds an image using Dockerfile located in a different folder
<code>docker tag [imageName][name:tag]</code>	Tag an existing image
<code>docker create volume [volumeName]</code>	Creates a new volume
<code>docker volume ls</code>	Lists the volumes
<code>docker volume inspect [volumeName]</code>	Display the volume info
<code>docker volume rm [volumneName]</code>	Deletes a volume

<code>docker volume prune</code>	Deletes all volumes not mounted
<code>docker compose build</code>	Build the images
<code>docker compose start</code>	Start the containers
<code>docker compose stop</code>	Stop the containers
<code>docker compose up -d</code>	Build and start
<code>docker compose ps</code>	List what's running
<code>docker compose rm</code>	Remove from memory
<code>docker compose down</code>	Stop and remove
<code>docker compose logs</code>	Get the logs
<code>docker compose exec [container] bash</code>	Run a command in a container
<code>docker compose --project-name test1 up -d</code>	Run an instance as a project
<code>docker compose -p test2 up -d</code>	Shortcut of [run as project]
<code>Docker compose ls</code>	List running projects
<code>docker compose cp [containerID]:[SRC_PATH] [DEST_PATH]</code>	Copy files from the container
<code>docker compose cp [SRC_PATH] [containerID]:[DEST_PATH]</code>	Copy files to the container

Examples:

Pull image and run container:

```
# pull and run an nginx server
docker run -dp 8080:8080 --name webserver nginx
```

-p 8080:8080 = Maps the host port to container

--name webserver = Container local name

-d = detached means that run in background

nginx = container name in the Docker registry

```
# list the running containers
```

```
docker ps
```

```
# Attach to the container
```

```
docker container exec -it webserver /bin/bash
```

```
# stop the container
```

```
docker stop webserver
```

webserver = the name of the container which was given when creating the container using docker run command

```
# remove container  
docker rm webserver
```

```
# remove the image  
docker rmi nginx
```

Build image from Dockerfile and run an app:

Dockerfile: A Dockerfile is a plain text document that contains a series of instructions and commands used by Docker to automatically build a container image. It acts as a blueprint or recipe, defining the step-by-step process for constructing a Docker image.

```
# build  
docker build -t webserver-image:v1 .
```

webserver-image:v1 = name:tag

```
# run  
docker run -d -p 8080:80 webserver-image:v1
```

```
# display  
curl localhost:8080
```

Example of a Dockerfile:

```
# Use an official Node.js runtime as a parent image
FROM node:18-alpine

# Set the working directory in the container
WORKDIR /app

# Copy package.json and package-lock.json to the working
directory
COPY package*.json ./

# Install application dependencies
RUN npm install

# Copy the rest of the application code
COPY . .

# Expose port 3000
EXPOSE 3000

# Define the command to run the application
CMD ["node", "app.js"]

# or you can use the following
# ENTRYPOINT ["node", "app.js"]
```

Persisting data- Mapping a volume:

```
# create a volume
docker volume inspect myvol

# inspect the volume
docker volume inspect myvol

# list the volumes
docker volume ls

# run a container with a volume
docker run -d --name devtest -v myvol:/app nginx:latest

-v myvol:/app = mapping the volume to a logical folder

# run a container with a volume
```

```
docker run -d --name devtest -v d:/test:/app nginx:latest
```

-v d:/test = instead of specifying a volume we can specify a local folder to map(not recommended for prod env)

```
# inspect the container
docker inspect devtest
```

Docker Compose:

```
# Build the service
docker compose build
```

```
# builds, (re)creates, starts, attaches to containers for
a service
docker compose up
```

```
# list the services
docker compose ps
```

```
# bring down what was created by UP
docker compose down
```

Note: The above commands should be executed in the same folder as project

```
# watch service logs in terminal
docker compose logs -f [serviceName]
```